

```

/**
 * Implementation of KalmanFilter class.
 *
 * @author: Hayk Martirosyan
 * @date: 2014.11.15
 */
#include <iostream>
#include <stdexcept>

#include "kalman.hpp"

```

```

KalmanFilter::KalmanFilter(
    double dt,
    const Eigen::MatrixXd& A,
    const Eigen::MatrixXd& C,
    const Eigen::MatrixXd& Q,
    const Eigen::MatrixXd& R,
    const Eigen::MatrixXd& P)
: A(A), C(C), Q(Q), R(R), P0(P),
  m(C.rows()), n(A.rows()), dt(dt), initialized(false),
  I(n, n), x_hat(n), x_hat_new(n)
{
    I.setIdentity();
}

```

```

KalmanFilter::KalmanFilter() {}

```

```

void KalmanFilter::init(double t0, const Eigen::VectorXd& x0) {
    x_hat = x0;
    P = P0;
    this->t0 = t0;
    t = t0;
    initialized = true;
}

```

```

void KalmanFilter::init() {
    x_hat.setZero();
    P = P0;
    t0 = 0;
    t = t0;
    initialized = true;
}

```

```

void KalmanFilter::update(const Eigen::VectorXd& y) {
    if(!initialized)
        throw std::runtime_error("Filter is not initialized!");

    x_hat_new = A * x_hat;
    P = A*P*A.transpose() + Q;
    K = P*C.transpose()*(C*P*C.transpose() + R).inverse();
    x_hat_new += K * (y - C*x_hat_new);
    P = (I - K*C)*P;
    x_hat = x_hat_new;

    t += dt;
}

```

```

void KalmanFilter::update(const Eigen::VectorXd& y, double dt, const Eigen::MatrixXd A) {
    this->A = A;
    this->dt = dt;
    update(y);
}

```